

Server2 bounded resolver operations

S07 internal library, [spec](#), [architecture](#). No production route, real provider or AWS connection is activated. Every example uses the fresh loopback peer with public test credentials.

Run the functional example

Build from the repository root, then launch the owned peer from server2/:

```
cargo build --manifest-path server2/Cargo.toml --release --examples
cd server2
python3 tests/storage/catalog_peer.py --smoke target/release/examples/resolver_smoke
```

The example seeds two-page raw acquisitions, resolves checked S3 data with provider calls disabled, checks byte-identical memory reuse and zero additional HTTP work, reads all scoped revisions, then cancels an initiating waiter during recorded acquisition. Another waiter receives the result only after complete publication. Drain rejects new admission, waits for the admitted owner, and reports no unfinished view jobs. A separate tiny cache demonstrates that an evicted pin remains charged until its last owner drops.

An assertion failure or nonzero exit is a failure. Local elapsed time is only fixture evidence, never AWS latency, authentication, quota or API parity. `catalog_peer.py --smoke` accepts this release example even though its help text names the older catalog example.

Integrate the library

Construct `CatalogBackend` with a bounded transport and default-or-smaller `CatalogLimits`. Create an immutable `ResponseKey` with every output-affecting semantic and a `ResolutionRequest` with an explicit finite period scope, selection, limits and monotonic TTL/refresh policy. Unknown parameters must be rejected by the future route mapper.

`FundedAcquirer` receives `AcquisitionRequest::resolution()` and `OperationContext::deadline, is_cancelled, check`. Return the exact ordered `RawRecord` pages and `GroupDeclaration`, or a terminal `resolver::Error`. S08 must durably reserve provider-unit funding before any HTTP attempt. The recorded example callback is not an implementation or proof of that funding.

`ViewBuilder` parses checked borrowed raw views and assembles into `CappedOutput`. Do not allocate unbounded temporary output and do not copy a precise request into archival bodies/identities.

`CheckedInput::require_full_history()` rejects selected acquisitions. The borrowed `Response::bytes()` API keeps response pins attached to their byte lease.

Coverage, recovery and freshness

Cold resolution repairs only caller-owned finite prefixes. `CompleteEmpty` authorizes acquisition only for that exact catalog/period scope, never global absence. Incomplete, corrupt, Capacity and Timeout are terminal recovery outcomes rather than permission to fetch. Indexed scopes proceed to bounded reading; a selected or published group is not full history.

Earliest/latest reads skip unrelated acquisitions, keeping all ordered pages of each selected group. `FullHistory` must reach the final cursor with stable dependencies; it means all revisions intersecting the owned scope. Assemblers extract the requested date/fields from complete pages. A page/deadline bound must not truncate absent-field merging. Exact targeted access remains available.

Responses are cached until the earlier TTL/refresh boundary. Refresh checks generation, ETag, version and absence tokens; `Resolver::invalidate()` conservatively invalidates known local changes. External revisions have bounded refresh lag. Known excluded groups never enter the response map because missing descriptors can appear without changing the catalog ETag. Transport/5xx refresh failure may use still-valid memory without extending its TTL; corruption cannot use that fallback.

New acquisitions complete raw, descriptor and sibling catalog publication before response assembly. Failure or uncertain publication does not return new success. Current routes must keep their legacy error/fallback behavior when these primitives are eventually wired.

Limits and lifetime

Default retained cache is 128 MiB across raw, parsed and response values. Four owners reserve 64 MiB each; waiter admission is 128. Values retired by eviction stay charged while callers hold them. Concrete insertions charge key/envelope/dependency metadata and payload. Generic `WeightedCache` callers must supply truthful total weights and independently bound mutable allocations.

Backend bounds include 1 MiB control documents, 8 MiB retained controls, 32 MiB output, 8 MiB raw bodies, at most 128 members/16 partitions/2,048 entries. Requests allow at most 64 whole-acquisition pages and 32 MiB materialized input including envelopes. Parsed output totals at most 8 MiB and response output defaults to 1 MiB. A callback must honor its `maximum_input_bytes()` limit before retaining the full response.

The view-builder pool defaults to two CPU permits. Dependency parsing and existing S05/S06 processing have separate bounded pools and stream limits. Two is not a global CPU limit. Started S07 blocking jobs retain operation leases even after timeout. Existing storage jobs retain their own bounds. These numbers are logical accounting, not hard process RSS; allocator overhead, adapter scratch and copies require real feasibility measurements.

The owner deadline starts at admission and cannot exceed three seconds. Each waiter keeps an independent deadline. Cancelling one waiter does not cancel shared work. Drain stops admission and awaits owners within its deadline; expiration signals cooperative cancellation. `DrainReport` counts unfinished owners and view-builder jobs, not every backend/storage codec worker. Tokio cannot forcibly abort started blocking work.

Metrics, cost and limitations

`Resolver::metrics()` reports response hits/misses/refreshes, shared flights, rejections, acquisition/publication counts, cancelled waiters, retained and retired/pending bytes, operation reservations, owners/waiters and view jobs. These are library snapshots; no new HTTP metrics route is wired.

The raw cache currently retains checked identities after S06 network reads, rather than bypassing GETs. Parsed acquisition results are reused, and warm response hits truly avoid S3/provider work. `FullHistory` uses page-size-one S06 traversal with repeated descriptor normalization and cold finite-prefix repair. No cold speed or asymptotic improvement is claimed; S09/O01 remain measurement/optimization gates.

Exact coordinate and address geocode keys are volatile only and reject archive authorization. Persistent coarse projection, actual labels, cross-month validity and route compatibility remain S10/late gates. No shared cache, disk serving database, persisted normalized view, live AWS/provider action or deployment is introduced. Merge is a separate user decision.

Failure investigation

Use sanitized typed errors and the above counters. Confirm the finite scope and selection before interpreting a miss. Capacity can reflect live retired pins or a started CPU job after owner timeout. Do not retry an Ambiguous publication as a new identity or hide incomplete recovery with a provider request. Preserve original diagnostics without credentials or precise request history.

The maintained tests run in default parallel mode and use milestone-controlled concurrency. Fixture startup or sandbox binding errors are setup failures, not passed tests or intended regression Red. The retained eviction assertion has an actual failure-to-pass transition; the additional release example supplies distinct HTTP functional evidence.

Actual local CLI usage

```
S07 - actual release HTTP smoke | loopback only

$ python3 tests/storage/catalog_peer.py --smoke target/release/examples/resolver_smoke

cold/warm: checked fixture bytes identical; warm S3/provider calls = 0
history: all scoped revisions retained; whole ordered pages preserved
singleflight/drain: cancelled initiator detached; one recorded acquisition; complete durable group before
response
pressure: evicted pin stays charged; last-owner drop releases capacity
metrics: hits=1, shared=1, acquisitions=1, publications=1, owners=0, cache_bytes=11366
PASS local real HTTP resolver smoke (81.33 ms); no AWS/provider/route parity claim
```